



Berachain: Internal Contracts Security Review

Cantina Managed review by:

Mustafa Hasan, Lead Security Researcher

Chinmay Farkya, Security Researcher

July 15, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Trust assumptions	3
3	Overview	4
4	Findings	5
4.1	Informational	5
4.1.1	<code>renounceRole()</code> should be overridden to revert	5
4.1.2	Recommended documentation changes	5
4.1.3	<code>isPegged()</code> function in <code>HoneyFactory</code> now skips staleness checks	5

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Jul 6th to Jul 7th the Cantina team conducted a review of `contracts-internal` on commit hash `0a1dc1e5`. The team identified a total of **3** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Gas Optimizations	0	0	0
Informational	3	2	1
Total	3	2	1

2.1 Trust assumptions

The trust model concentrates power in a few actors: oracle governance controls the feeds and can upgrade the implementation, the RedStone adapter and its signers are trusted as both price source and staleness/positivity guarantor, and the Honey manager can swap the entire oracle via `setPriceOracle`. Review attention centered on the peg gate's use of the unchecked getter and its interaction with feed freshness, the hardcoded decimal scaling, the asymmetry between the unsafe and strict getters, and the deterministic deployment not binding the deployer or init parameters into the `CREATE2` address.

3 Overview

This review covered the changes in the `main...feat/redstone-price-oracle` diff — the new `RedStone` price oracle, its deterministic deployer, and the accompanying `HoneyFactory` integration — and did not extend to the surrounding, unmodified system beyond what was needed to reason about those changes. The change set adds `RedStonePriceOracle`, a UUPS-upgradeable, access-controlled `IPriceOracle` implementation that reads from a `RedStone` adapter, keeps a per-asset `dataFeedIds` mapping, and rescales feed values from a hardcoded 8 decimals to 18-decimal WAD. A single `DEFAULT_ADMIN_ROLE` held by governance gates both `setDataFeedId` and the UUPS upgrade, giving that role full control over the feeds in use and the implementation itself. The oracle is deployed by `RedStonePriceOracleDeployer`, which stands up the implementation and an `ERC1967Proxy` through the Arachnid `CREATE2` factory and initializes them atomically in its constructor.

The oracle offers a checked read path (`getPrice` , which enforces the adapter's 30-hour staleness and positive-value checks) and unchecked ones (`getPriceUnsafe` , `getPriceNoOlderThan` , and `priceAvailable`). `HoneyFactory` consumes it in only two ways: the `isPegged` peg gate, which uses the unchecked `getPriceUnsafe` and drives `isBasketModeEnabled` for mint, redeem, `addCollateral` , and `recapitalize` ; and `_getPrice` , which uses the strict `getPrice` and is reached only by `liquidate` .

No high or medium severity issues were identified in the in-scope code. We thank the Berachain team for their continued efforts and collaboration throughout the engagement.

4 Findings

4.1 Informational

4.1.1 `renounceRole()` should be overridden to revert

Severity: Informational.

Context: `RedStonePriceOracle.sol`#L4.

Description: `RedStonePriceOracle.sol` inherits from OpenZeppelin's `AccessControlUpgradeable` library.

The `renounceRole()` function allows an address to give up its access rights. If this function is called by the `DEFAULT_ADMIN_ROLE` holder by mistake, admin access can be lost permanently.

Recommendation: Consider overriding `renounceRole()` to always revert.

Berachain: Acknowledged.

Cantina Managed: Acknowledged.

4.1.2 Recommended documentation changes

Severity: Informational.

Context: `IPriceOracle.sol`#L5, `RedStonePriceOracle.sol`#L118.

Description: The following changes are recommended:

- The natspec for `IPriceOracle` interface says `GenericinterfacethatwrapsthePythoracle` ⇒ It should be changed to include Redstone Oracle as well.
- The comment in `getPriceNoOlderThan()` function says `Throwspanicrevertifageisgreaterthanb`
`lock.timestamp` ⇒ This is incomplete because apart from the panic revert, there is another explicit revert condition that is not documented currently.

Recommendation: Consider modifying the comments as mentioned.

Berachain: Fixed in PR 116.

Cantina Managed: Fix verified.

4.1.3 `isPegged()` function in `HoneyFactory` now skips staleness checks

Severity: Informational.

Context: `HoneyFactory.sol`#L565-L571.

Description: This was the `isPegged()` logic before the changes made to `HoneyFactory` :

```
function isPegged(address asset) public view returns (bool) {
    if (!priceOracle.priceAvailable(asset)) {
        return false;
    }
    IPriceOracle.Data memory data = priceOracle.getPriceUnsafe(asset);
    if (data.publishTime < block.timestamp - priceFeedMaxDelay) {
        return false;
    }
    return (1e18 - lowerPegOffsets[asset] <= data.price) && (data.price <= 1e18 +
    ↪ upperPegOffsets[asset]);
}
```

After the `PythOracle` setup was replaced with `RedStoneOracle`, the `priceFeedMaxDelay` check was removed from this function. Now it relies on `getPriceUnsafe` for price info, but the `RedStonePriceOracle` calls `getLastUpdateDetailsUnsafe()` on the Redstone adapter.

This call flow completely skips staleness checks and positive-value checks now.

Recommendation: If this change in `isPegged()` is intended, consider documenting the assumptions.

Berachain: Staleness of the price feed is monitored off-chain. This has been documented in [PR 116](#).

Cantina Managed: Fix verified.